

# Knuth-Morris-Pratt Algorithm

Christian Rinderknecht

September 2026

## Basic definitions

An **alphabet** is a non-empty, finite set of symbols, called **letters**, for example 'a', 'b' etc.

A **word** is a finite series of letters from an alphabet, like abracadabra. A letter is a word.

Just as letters can be joined to make up words, so can words: the word  $u \cdot v$  is made of the letters of word  $u$  followed by the letters of word  $v$ , for instance, if  $u = \text{back}$  and  $v = \text{up}$ , then  $u \cdot v = \text{backup}$ . This operation is called **concatenation**.

Concatenation is associative:  $(u \cdot v) \cdot w = u \cdot (v \cdot w)$ . The operator may be omitted:  $(uv)w = u(vw)$ . Concatenation is a non-commutative product, so it has a neutral element  $\varepsilon$ , called the **empty word**:

$$u \cdot \varepsilon = \varepsilon \cdot u = u.$$

## Basic definitions

We denote the **repetition** of a word with an exponent, e.g.  $a^3 = aaa$ . The **length** of a string  $x$  is the length of the associated sequence and is noted  $|x|$ . So  $|\varepsilon| = 0$ ,  $|a| = 1$  and  $|a^3| = 3$ .

A word  $x$  is a **factor** of a word  $y$  if there exists two words  $u$  and  $v$  (possibly empty) such that  $y = uxv$ . In other words,  $u$  occurs in  $y$ .

The word  $x$  is a **prefix** of  $y$ , noted  $x \leqslant y$ , if  $u = \varepsilon$ , that is, if  $y = xv$ . Moreover, it is a **proper prefix**, noted  $x < y$ , if  $v \neq \varepsilon$ .

Given  $y = uxv$ , the word  $x$  is a **suffix** of  $y$  if  $v = \varepsilon$ . Furthermore, it is a **proper suffix** if  $u \neq \varepsilon$ .

Let  $a$  range over letters, and  $x$  and  $y$  over words. The prefix relation is easy to define inductively:

- $\forall y. \varepsilon \leqslant y$ ;
- $\forall a, x, y. x \leqslant y \Rightarrow a \cdot x \leqslant a \cdot y$ .

## Basic definitions

A letter in a word can be uniquely characterised by a natural number, called **index**, assuming that the first letter has index 0. Read <https://www.cs.utexas.edu/~EWD/transcriptions/EWD08xx/EWD831.html>

For example, if  $x = \text{top}$ , then the letter at index 0 is written  $x[0] = \text{t}$  and the one at index 2 is  $x[2] = \text{p}$ .

A factor  $x$  of  $y$  can be identified by the index of  $x[0]$  in  $y$ .

The end of the factor can also be given; for example,  $x = \text{sit}$  is a factor of  $y = \text{curiosity}$  at index 5, written  $y[5, 7] = x$ . This is sometimes called a **slice** in some programming languages.

Sometimes the upper bound can be left unspecified:  $y[i \dots]$ , meaning  $y[i, |y| - 1]$ , if  $i < |y|$ . That is,  $y[i \dots]$  is a **suffix** of  $y$ . Likewise:  $y[\dots i]$  is a **prefix** of  $y$ .

# The prefix predicate

An imperative definition:

```
    PREFIX( $x, t$ )  
1   $i \leftarrow 0$   
2  while  $i < |x|$  and  $i < |t|$   
3      do if  $x[i] = t[i]$   
4          then  $i \leftarrow i + 1$   
5          else  $i \leftarrow |x| + 1$   
6  PREFIX  $\leftarrow i = |x|$ 
```

Does it terminate? What test cases would be meaningful? Is it correct?  
Is it optimal?

# The prefix predicate

A single static assignment definition:

$\text{PREFIX}(x, t)$

1  $\text{PREFIX} \leftarrow \text{PREFIX}'(0, x, t)$

$\text{PREFIX}'(i, x, t)$

1 **if**  $i < |x|$  **and**  $i < |t|$

2     **then if**  $x[i] = t[i]$

3         **then**  $\text{PREFIX}'(i + 1, x, t)$

4         **else**  $\text{PREFIX}'(|x| + 1, x, t)$

5     **else**  $\text{PREFIX}' \leftarrow i = |x|$

## Naïve search

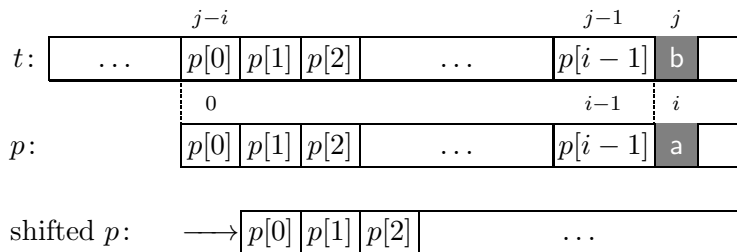
Factor matching is common in text editing, although it is usually better known as **exact string matching** in the academic field of *text algorithmics* (sometimes called *stringology*).

Because of the asymmetric nature of the search, the word  $p$  to find will from now on be called the **pattern**, and the word  $t$  the **text**.

Assuming  $p$  and  $t$  are not empty, the naïve search simply consists in checking whether  $p$  is a prefix of  $t$ : if so, we are done, otherwise, the pattern is shifted by one letter to the right and, again, it is checked whether it is a prefix of that text suffix.

If  $p$  cannot be shifted anymore because its end would surpass the end of  $t$ , then it is not a factor: the search failed.

# Naïve search



Naïvely matching pattern  $p$  against text  $t$  (failure in grey), where  $p[i] \neq t[j]$  (the letters 'a' and 'b' are not relevant in themselves).

# Naïve search

An imperative program:

```
    NAIVE( $p, t$ )
1   $i \leftarrow 0; j \leftarrow 0$ 
2  while  $i < |p|$  and  $j < |t|$ 
3      do if  $t[j] = p[i]$ 
4          then  $j \leftarrow j + 1; i \leftarrow i + 1$ 
5          else  $j \leftarrow j + 1 - i; i \leftarrow 0$     ▷ We slide  $p$  by 1.
6   $\text{NAIVE} \leftarrow i = |p|$ 
```

What test cases would be meaningful?

Would it be clearer to use two loops?

How about a version with single static assignments and slices? Without slices?

## Naïve search

The input is a pattern  $p$  of length  $m$  and a text  $t$  of length  $n$

What configuration of the input leads to the minimal number of comparisons? What is that number?

In the worst case,  $p$  is not in  $t$  and the test at line 3 always fails on the last letter of  $p$ , leading to make  $p$  slide of one position at line 5 the latest as possible.

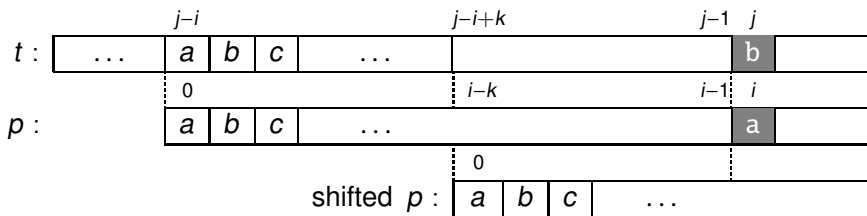
An example of positive match in the worst case is  $p = a^{m-1}b$  and  $t = a^{n-1}b$ , with  $m \leq n$ .

There are  $n - m + 1$  positions in  $t$  to compare with the first letter of  $p$ , and since there are  $m$  letters in  $p$ , it makes a total of  $(n - m + 1)m = nm - m^2 + m < |t| \times |p|$  positions.

What about the worst case when there is no match?

# Morris-Pratt algorithm

In case of mismatch, the naïve algorithm resumes comparing the first letters of  $p$  without using the information of the partial success, to wit, we know  $p[0, i-1] = t[j-i, j-1]$  and  $p[i] \neq t[j]$ .

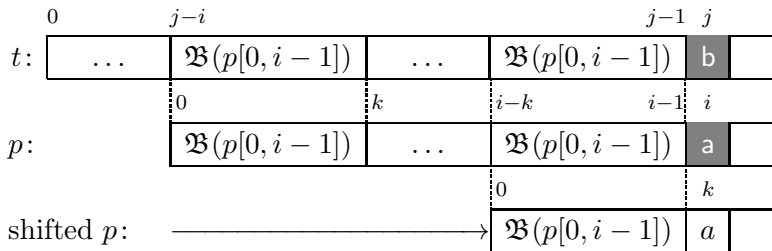


If a new search starts in  $t$  at position  $j-i+k$ , with  $1 \leq k \leq i$ , it compares a prefix of  $p$  to  $t[j-i+k \dots j-1]$ . Since  $t[j-i+k \dots j-1] = p[i-k, i-1]$ , it compares a prefix of  $p$  to a suffix of  $p[0 \dots i-1]$ , that is, to a part of itself!

## Morris-Pratt algorithm

The important point here is that these comparisons are *independent* of the text  $t$ , since they apply to part of the pattern  $p$  itself.

Contrary to naïve factoring, letters in the text are compared in a strictly increasing order (never having to backtrack).



where  $\mathfrak{B}(p)$  is the **maximum border** of  $p$ : the longest prefix of  $p$  that is also a suffix of  $p[0 \dots i-1]$ .

## Morris-Pratt algorithm

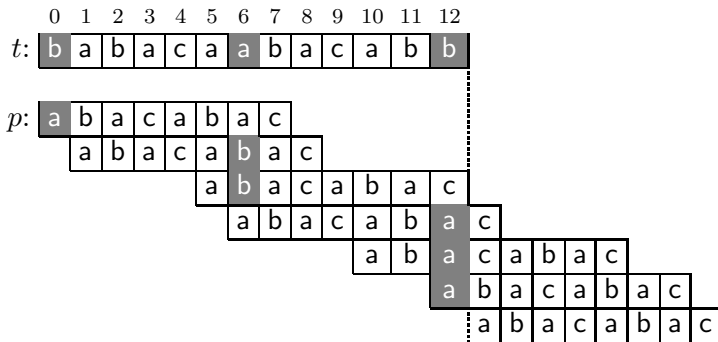
The strategy devised here is to **compute the maximum borders of all prefixes of  $p$  before we start the search** in the text, and then use this information to avoid repeating the same comparisons at different positions in the text.

This **preprocessing** on  $p$  will indeed allow to recognise the sole configurations where the search may succeed.

In other words, we need to understand borders.

## Morris-Pratt algorithm

But before that, let us consider an example where, in the end,  $p$  is not found to be a factor of  $t$ . As usual, letters on a grey background correspond to mismatches. It is clear that  $\mathcal{B}(a) = \varepsilon$ , for all letters  $a$ .



(The text is not shown in its entirety.)

## Morris-Pratt algorithm

The **border** of a non-empty word  $y$  is a prefix of  $y$  which is also a suffix.

To understand the case of the **empty border**, consider  $\mathfrak{B}(\text{abac}) = \varepsilon$ , simply because we have  $\text{abac} = \underline{\varepsilon}\text{abac}\underline{\varepsilon}$ .

For example, the word 'abacaba' has three borders:  $\varepsilon$ , 'a' and 'aba'. The last one is the **maximum border** and we write  $\mathfrak{B}(\underline{\text{abacaba}}) = \text{aba}$ .

Note that **borders can overlap**; for example,  $\mathfrak{B}(\underline{\text{aaaa}}) = \mathfrak{B}(\underline{\text{aaaa}}) = \text{aaa}$ .

## Morris-Pratt algorithm

As often when working at the level of specifications, we seek an inductive definition for the function at hand, in this instance, the maximum border of a word.

We then have the choice of finding either  $\mathfrak{B}(ay)$  or  $\mathfrak{B}(ya)$ , with  $y \neq \varepsilon$ , assuming that we know  $\mathfrak{B}(y)$ .

Since we are interested in knowing the maximum borders of all the *prefixes* of a given pattern, the  $ya$  is more suitable ( $y < ya$ ).

The idea is to recursively consider  $\mathfrak{B}(y) \cdot a$ :

- if it is a prefix of  $y$ , then  $\mathfrak{B}(ya) = \mathfrak{B}(y) \cdot a$ ;
- otherwise, we consider the border of  $\mathfrak{B}(y) \cdot a$ .

# Morris-Pratt algorithm

Consider the following examples where  $y$  and  $\mathfrak{B}(y)$  are given:

$$y = \text{abaabb}, \quad \mathfrak{B}(y) = \varepsilon, \quad \mathfrak{B}(y \cdot b) = \mathfrak{B}(\mathfrak{B}(y) \cdot b) = \mathfrak{B}(b) = \varepsilon;$$

$$y = \text{baaaba}, \quad \mathfrak{B}(y) = \text{ba}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(y) \cdot a = \text{baa};$$

$$y = \text{abbbab}, \quad \mathfrak{B}(y) = \text{ab}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}^2(y) \cdot a = a.$$

For example,  $\mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^2(y) \cdot a) = \mathfrak{B}^3(y) \cdot a$ :

$$y \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}(y) & b & \dots & \mathfrak{B}(y) & a \\ \hline \end{array}$$

$$\mathfrak{B}(y) \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^2(y) & c & \dots & \mathfrak{B}^2(y) & a \\ \hline \end{array}$$

$$\mathfrak{B}^2(y) \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^3(y) & a & \dots & \mathfrak{B}^3(y) & a \\ \hline \end{array}$$

# Morris-Pratt algorithm

Formally, for all words  $y \neq \varepsilon$  and any letter  $a$ ,

$$\mathfrak{B}(a) := \varepsilon; \quad \mathfrak{B}(y \cdot a) := \begin{cases} \mathfrak{B}(y) \cdot a, & \text{if } \mathfrak{B}(y) \cdot a \leqslant y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \text{otherwise.} \end{cases}$$

Unfolding the equations yields:

$$\begin{array}{ll} \mathfrak{B}(ya) = \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \mathfrak{B}(y) \cdot a \not\leqslant y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^2(y) \cdot a), & \mathfrak{B}^2(y) \cdot a \not\leqslant \mathfrak{B}(y); \\ \vdots & \vdots \\ \mathfrak{B}(\mathfrak{B}^{p-1}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^p(y) \cdot a), & \mathfrak{B}^p(y) \cdot a \not\leqslant \mathfrak{B}^{p-1}(y); \end{array}$$

and  $\varepsilon \notin \{y, \mathfrak{B}(y), \dots, \mathfrak{B}^{p-1}(y)\}$ .

## Morris-Pratt algorithm

Therefore, an equivalent definition is:

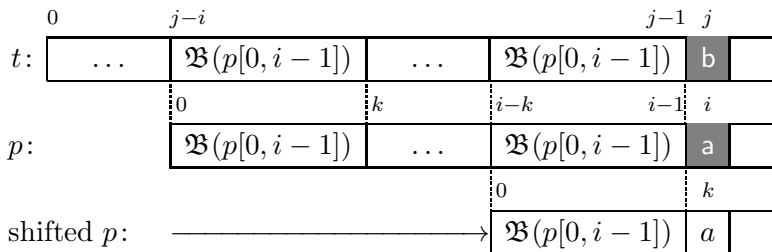
$$\mathfrak{B}(ya) = \begin{cases} \mathfrak{B}^q(y) \cdot a, & \text{if } \mathfrak{B}^q(y) \cdot a \leqslant y; \\ \varepsilon, & \text{otherwise;} \end{cases}$$

with the additional constraint that  $q$  **must be as small as possible** (so we define the maximum border).

This form of the definition of  $\mathfrak{B}$  is simpler because it does not contain an embedded call.

## Morris-Pratt algorithm

We do not actually need to compute the maximum borders of all the prefixes of the pattern. Let us have a look back at



What we need is the offset corresponding to the **length of the maximum border** being considered ( $k$ , in the figure).

# Morris-Pratt algorithm

Let us define a function  $\mathfrak{F}_x$  on all the prefixes of a word  $x$  as follows:

$$\mathfrak{F}_x(|y|) := |\mathfrak{B}(y)|, \text{ for all } x \text{ and } y \neq \varepsilon \text{ such that } y \leq x. \quad (1)$$

For reasons which will be clear soon, this function is called the **failure function** of  $x$ . An equivalent definition is

$$\mathfrak{F}_x(i) = |\mathfrak{B}(x[\dots i - 1])|, \text{ for all } x \text{ and } i \text{ such that } 0 < i \leq |x|.$$

Let's take the equations defining of the maximum borders and take the lengths on both sides:

$$|\mathfrak{B}(ya)| = \begin{cases} |\mathfrak{B}^q(y) \cdot a| = 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ |\varepsilon| = 0, & \text{otherwise.} \end{cases}$$

## Morris-Pratt algorithm

If  $ya \leq x$ , then  $|\mathfrak{B}(ya)| = \mathfrak{F}_x(|ya|) = \mathfrak{F}_x(|y| + 1)$ . Let  $i := |y| > 0$ .

$$\mathfrak{F}_x(i + 1) = \begin{cases} 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

Let's set aside for a moment  $\mathfrak{B}^q(y) \cdot a \leq y$  and return to the definition of  $\mathfrak{F}$ , from which we deduce:

$$\mathfrak{F}_x^q(|y|) = |\mathfrak{B}^q(y)|, \text{ with } y \leq x,$$

which we can prove by complete induction on  $q$ . Let us call this property  $P(q)$ . Trivially, we have  $P(0)$ . Let us suppose  $P(n)$  for all  $n \leq q$ : this is the induction hypothesis. We suppose  $y \leq x$  and prove now  $P(q + 1)$ :

$$\mathfrak{F}_x^{q+1}(|y|) = \mathfrak{F}_x^q(\mathfrak{F}_x(|y|)) = \mathfrak{F}_x^q(|\mathfrak{B}(y)|) \doteq |\mathfrak{B}^q(\mathfrak{B}(y))| = |\mathfrak{B}^{q+1}(y)|,$$

where  $(\doteq)$  is a valid application of the induction hypothesis because  $\mathfrak{B}(y) < y \leq x$ . This proves  $P(q + 1)$  and the induction principle entails that  $P(n)$  holds for all  $n \geq 0$ .

## Morris-Pratt algorithm

So

$$\mathfrak{F}_x(i+1) = \begin{cases} 1 + \mathfrak{F}_x^q(i), & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

There is one part of the definition that we did not use:  $\mathfrak{B}(a) := \varepsilon$ . It implies  $\mathfrak{F}_x(1) = \mathfrak{F}_x(|a|) = |\mathfrak{B}(a)| = |\varepsilon| = 0$  and, since the definition of  $\mathfrak{F}$  implies  $\mathfrak{F}_x(1) = 1 + \mathfrak{F}_x(0)$ , so  $\mathfrak{F}_x(0) = -1$ .

We can now return to the last missing bit:

$$\mathfrak{B}^q(y) \cdot a \leq y \text{ and } ya \leq x \text{ and } |y| = i.$$

It is equivalent to the following:

$$y[|\mathfrak{B}^q(y)|] = a \Leftrightarrow y[\mathfrak{F}_x^q(i)] = a \Leftrightarrow x[\mathfrak{F}_x^q(i)] = x[|y|] \Leftrightarrow x[\mathfrak{F}_x^q(i)] = x[i].$$

## Morris-Pratt algorithm

Gathering everything:

$$\mathfrak{F}_x(0) = -1 \quad \text{and} \quad \mathfrak{F}_x(i+1) = \begin{cases} 1 + \mathfrak{F}_x^q(i), & \text{if } x[\mathfrak{F}_x^q(i)] = x[i]; \\ 0, & \text{otherwise;} \end{cases}$$

where  $q$  is the smallest nonzero natural satisfying the condition.

This can be further simplified into

$$\mathfrak{F}_x(0) = -1 \quad \text{and} \quad \mathfrak{F}_x(i+1) = 1 + \mathfrak{F}_x^q(i), \quad \text{with } i \geq 0,$$

and  $q > 0$  is the smallest natural such that  $\mathfrak{F}_x^q(i) = -1$  or  $x[\mathfrak{F}_x^q(i)] = x[i]$ .